

The lux `apply_patch` Atomicity Boundary under Reentrant, Order-Independent Notification: a Z Specification

Formal model of the all-or-nothing patch discipline with nested patches

July 2026

1 Introduction

An agent patches a single Element-ABC element by sending a mapping of field names to new values. `Element.apply_patch` applies the entries in order, each through a `_set_<key>` setter, and promises *all-or-nothing*: if any setter rejects its value, every field the patch already touched is restored and the element is left exactly as it was before the patch began. A composed table also has an *observer* — the filtered table’s Hub-side model — which folds a selection write into a full selection spanning filter-hidden rows, and folds an external *rows* write in as its new dataset. The property that must hold is that a rejected patch produces no observable change at all, that a successful patch leaves the model reflecting the patch’s final values, and that the model’s fold does not depend on the order in which one patch’s notifications happen to fire.

The same class of defect recurred across four fix/review rounds:

1. the table’s selection lived in a *mutable* composed model, so the shallow `dict(vars(self))` snapshot captured the model *reference*; a setter mutated it in place, and rollback restored a reference to an already-mutated model;
2. the selection setter notified observers *inside* the patch, so the model folded an intermediate selection before a later key was rejected; rollback restored the fields but could not un-fire the notification;
3. the model absorbed an external rows refresh as its dataset and re-projected by issuing a *nested* `apply_patch` from inside the outer patch’s commit-phase notification; without a guard the model folds its own filtered subset back in as the dataset, unbounded;
4. one `apply_patch` that set *both* rows and selection raced: the commit flushed the rows notification first, the model re-projected off a *stale* full selection and overwrote the freshly-set selection, and the later selection notification read the clobbered value.

Round 1 was closed by making the composed model immutable; round 2 by deferring notification to commit; round 3 by giving each `apply_patch` its own buffer (reset to `None` on entry) and a `_reprojecting` boolean that excludes the model’s own inner write from re-absorption; round 4 by making the rows-absorb *batch-aware* — it folds the element’s *committed* selection before re-projecting, so reading the element’s post-patch state makes the fold independent of which notification the commit flushes first. All four fixes survive here as fidelity variants (Section 11) that reintroduce each defect to confirm the model still detects it. Empirical round-by-round fixing kept missing an interleaving, so this document models the boundary as a finite state machine over a bounded *stack* of patch frames whose commit delivers buffered notifications in any order, letting ProB enumerate every patch shape, nesting order, and intra-batch delivery order exhaustively.

2 Basic Types

Because the reentrancy is tied to specific fields (a *rows* write triggers a re-projection; a *selected_row_ids* write only folds), the carriers are modelled concretely as free types rather than as unconstrained given sets. This bakes in exactly the topology under test and keeps every carrier finite, so ProB is exhaustive without a `DEFAULT_SETSIZE` parameter.

$FMODE ::= setting \mid flushing$

A patch frame is **setting** while its setters run (buffering notifications) and **flushing** while its committed notifications are delivered.

$SWITCH ::= \text{off} \mid \text{on}$

Used for the `_reprojecting` guard and for a frame’s “is a re-projection” marker.

$GOODVAL ::= a \mid b$

The values a field may validly hold. Two distinct values are enough to show a field *changing* and to distinguish an agent’s pushed value (**b**, say, a fresh dataset or a patched selection) from a stale or subset value (**a**), so that “the model absorbed its own subset” and “the re-projection clobbered the patched selection” are both observable.

$KEY ::= \text{krows} \mid \text{kssel}$

The two fields under test: **krows** (the element’s rows/dataset) and **kssel** (the element’s selection).

A patch step either sets a known field to a valid value or names a value the setter rejects:

$STEP ::= \text{setK} \langle\langle KEY \times GOODVAL \rangle\rangle \mid \text{badK} \langle\langle KEY \rangle\rangle$

setK: set field **k** to valid value **g** (the setter succeeds). **badK**: set field **k** to a rejected value (the setter raises, triggering rollback). Each constructor is an injection and their ranges partition **STEP**, so the test *head* $q \in \text{ran setK}$ distinguishes them. Two projections recover a step’s target and (for a set) its value:

$$\left| \begin{array}{l} \text{stepKey} : STEP \rightarrow KEY \\ \text{stepVal} : STEP \rightarrow GOODVAL \end{array} \right| \begin{array}{l} \forall k : KEY; g : GOODVAL \bullet \\ \quad \text{stepKey}(\text{setK}(k, g)) = k \wedge \text{stepVal}(\text{setK}(k, g)) = g \\ \forall k : KEY \bullet \text{stepKey}(\text{badK } k) = k \\ \text{dom stepVal} = \text{ran setK} \end{array}$$

The queue bound keeps every patch (agent-built or re-projection-generated) finite, and the stack bound caps the reentrancy depth so the carrier is finite:

$$\left| \begin{array}{l} \text{maxlen} : \mathbb{N} \\ \text{maxdepth} : \mathbb{N} \end{array} \right| \begin{array}{l} \text{maxlen} = 2 \\ \text{maxdepth} = 3 \end{array}$$

A patch of length two exhibits “set a watched field, then fail on a second” and the dual **{rows, selection}** patch; a re-projection writes exactly two keys (**krows** then **kssel**). A stack of depth two is the deepest the correct design reaches (an outer patch and its one nested re-projection); depth three gives ProB room to exhibit the unbounded self-recursion the guard prevents.

3 The Patch Frame Stack

Reentrancy means more than one **apply_patch** is live at once: the outer patch is mid-commit, delivering a notification, when the model starts a nested patch. Each live call is a *frame* with its own rollback snapshot, its own notification buffer, and (once committing) its own set of notifications still to deliver. A frame is the Z counterpart of one Python **apply_patch** stack activation.

The stack of frames is modelled as five parallel sequences kept in lockstep, one per frame attribute, rather than a sequence of a composite frame type — ProB enumerates a composite type’s unbounded sequence field as an unconstrained **ANY**, so the flat encoding is what keeps the search finite. Frame i is the tuple (*smode* i , *squeue* i , *ssnap* i , *sprops* i , *sreproj* i), and the top (innermost live) frame is at index $\#smode$:

- **smode** — **setting** while the frame’s setters run, **flushing** while its committed notifications are delivered.

- **squeue** — the frame’s unprocessed setters.
- **ssnap** — its rollback snapshot, a value copy of the fields as they were when the frame began.
- **sprops** — the props this frame has flagged. While **setting** this is the per-patch notification *buffer*; on commit the same set becomes the props still to *deliver*, and each is delivered in any order — modelling that a race over delivery order must not change the outcome.
- **sreproj** — on for a frame that is the model’s own re-projection, whose lifecycle raises and lowers the guard.

4 State

Alongside the stack, the state carries the element’s field values and four model scalars. **dataset** is the model’s absorbed dataset; **msel** is the model’s full selection. **extern** and **externsel** track the last *external* (agent, non-re-projection) rows and selection an outer patch committed — the values the agent actually pushed, which the model’s dataset and selection must equal. The predicate carries only the lockstep and length bounds; the atomicity property (Section 9) is deliberately *absent*, so a violating state stays reachable and ProB can find it.

Sys

```

smode : seq FMODE
squeue : seq(seq STEP)
ssnap : seq(KEY → GOODVAL)
sprops : seq(ℙ KEY)
sreproj : seq SWITCH
build : seq STEP
fields : KEY → GOODVAL
msel : GOODVAL
dataset : GOODVAL
extern : GOODVAL
externsel : GOODVAL
guard : SWITCH

```

```

#smode = #squeue ∧ #smode = #ssnap
∧ #smode = #sprops ∧ #smode = #sreproj
#smode ≤ maxdepth
#build ≤ maxlen
∀ i : dom squeue • #(squeue i) ≤ maxlen

```

The element is *idle* exactly when the stack is empty ($\#smode = 0$). **build** is the outer patch an agent assembles one step at a time while idle; building step-by-step keeps every input a single **STEP** over the finite **STEP** carrier, so ProB enumerates patch shapes without enumerating arbitrary sequences. The element’s rows/dataset field (**fields krows**) and the model’s dataset (**dataset**) diverge under a filter — the element shows the visible subset while the model holds the full dataset — which is why the dataset invariant compares **dataset** to **extern**, not to **fields krows**. **guard** is the **reprojecting** boolean.

5 Initialization

Init
Sys'

```

smode' = ⟨⟩ ∧ squeue' = ⟨⟩
∧ ssnap' = ⟨⟩ ∧ sprops' = ⟨⟩
∧ sreproj' = ⟨⟩ ∧ build' = ⟨⟩
fields' = {krows ↦ a, ksel ↦ a}
msel' = a ∧ dataset' = a
extern' = a ∧ externsel' = a
guard' = off

```

A single initial state: no patch in progress, every field holds **a**, the model's dataset and selection are **a** (consistent with the fields), the last external rows and selection were **a**, and the guard is down.

6 Beginning a Patch

An agent assembles the outer patch one step at a time while idle, appending each step to **build** up to the length bound. Any step is admissible, including a **badK** that will later trigger a rollback:

<i>Enqueue</i>	_____
ΔSys	
$p? : STEP$	
$\#smode = 0$	
$\#build < maxlen$	
$build' = build \hat{\ } \langle p? \rangle$	
$smode' = smode \wedge squeue' = squeue$	
$\wedge ssnap' = ssnap \wedge sprops' = sprops \wedge sreproj' = sreproj$	
$fields' = fields \wedge msel' = msel \wedge dataset' = dataset$	
$\wedge extern' = extern \wedge externsel' = externsel \wedge guard' = guard$	

BeginOuter applies the assembled patch: it pushes one frame onto each stack sequence with **squeue** taking the built steps, takes the rollback snapshot, and clears **build**. The empty patch is admissible:

<i>BeginOuter</i>	_____
ΔSys	
$\#smode = 0$	
$smode' = smode \hat{\ } \langle setting \rangle$	
$squeue' = squeue \hat{\ } \langle build \rangle$	
$ssnap' = ssnap \hat{\ } \langle fields \rangle$	
$sprops' = sprops \hat{\ } \langle \emptyset \rangle$	
$sreproj' = sreproj \hat{\ } \langle off \rangle$	
$build' = \langle \rangle$	
$fields' = fields \wedge msel' = msel \wedge dataset' = dataset$	
$\wedge extern' = extern \wedge externsel' = externsel \wedge guard' = guard$	

7 Applying a Patch

Each step processes the head of the top frame's queue. A successful set installs the value into the element and *buffers* the notification for that prop into the top frame's own **sprops** entry — it does not fire it:

<i>ApplyStepTop</i>	_____
ΔSys	
$\#smode \neq 0$	
$smode (\#smode) = setting \wedge squeue (\#smode) \neq \langle \rangle$	
$head (squeue (\#smode)) \in \text{ran setK}$	
$fields' = fields \oplus$	
$\{ stepKey (head (squeue (\#smode))) \mapsto stepVal (head (squeue (\#smode))) \}$	
$squeue' = squeue \oplus \{ \#smode \mapsto tail (squeue (\#smode)) \}$	
$sprops' = sprops \oplus$	
$\{ \#smode \mapsto (sprops (\#smode) \cup \{ stepKey (head (squeue (\#smode))) \}) \}$	
$smode' = smode \wedge ssnap' = ssnap \wedge sreproj' = sreproj$	
$msel' = msel \wedge dataset' = dataset \wedge extern' = extern$	
$\wedge externsel' = externsel \wedge guard' = guard \wedge build' = build$	

When the top frame's queue is empty it commits: it flips to **flushing** and its buffered **sprops** become its delivery set. An outer patch records the values it wrote to **krows** and **kset** as the new

external dataset and selection; a re-projection frame records neither, because its writes are the filtered subset and projection, not an agent push:

<i>CommitTopOuter</i>	<i>ΔSys</i>
$ \begin{aligned} &\#smode \neq 0 \\ &smode(\#smode) = setting \wedge squeue(\#smode) = \langle \rangle \\ &sreproj(\#smode) = off \\ &(krows \in sprops(\#smode) \Rightarrow extern' = fields\ krows) \\ &\wedge (krows \notin sprops(\#smode) \Rightarrow extern' = extern) \\ &(ksel \in sprops(\#smode) \Rightarrow externsel' = fields\ ksel) \\ &\wedge (ksel \notin sprops(\#smode) \Rightarrow externsel' = externsel) \\ &smode' = smode \oplus \{\#smode \mapsto flushing\} \\ &squeue' = squeue \wedge ssnap' = ssnap \\ &\wedge sprops' = sprops \wedge sreproj' = sreproj \\ &fields' = fields \wedge msel' = msel \wedge dataset' = dataset \\ &\wedge guard' = guard \wedge build' = build \end{aligned} $	

<i>CommitTopReproj</i>	<i>ΔSys</i>
$ \begin{aligned} &\#smode \neq 0 \\ &smode(\#smode) = setting \wedge squeue(\#smode) = \langle \rangle \\ &sreproj(\#smode) = on \\ &smode' = smode \oplus \{\#smode \mapsto flushing\} \\ &squeue' = squeue \wedge ssnap' = ssnap \\ &\wedge sprops' = sprops \wedge sreproj' = sreproj \\ &fields' = fields \wedge msel' = msel \wedge dataset' = dataset \\ &\wedge extern' = extern \wedge externsel' = externsel \\ &\wedge guard' = guard \wedge build' = build \end{aligned} $	

A rejected step aborts the top frame: its fields are restored from its own snapshot, its buffer is discarded unfired, and the frame is popped off every stack sequence. If the aborted frame was a re-projection, popping it lowers the guard — the model’s `try/finally` lowers `_reprojecting` whether the nested patch commits or raises. The outer frame beneath is untouched, so its delivery set survives and it resumes flushing:

<i>AbortReproj</i>	<i>ΔSys</i>
$ \begin{aligned} &\#smode \neq 0 \\ &smode(\#smode) = setting \wedge squeue(\#smode) \neq \langle \rangle \\ &head(squeue(\#smode)) \in \text{ran } badK \wedge sreproj(\#smode) = on \\ &fields' = ssnap(\#smode) \\ &smode' = front\ smode \wedge squeue' = front\ squeue \\ &\wedge ssnap' = front\ ssnap \wedge sprops' = front\ sprops \\ &\wedge sreproj' = front\ sreproj \\ &guard' = off \wedge msel' = msel \wedge dataset' = dataset \\ &\wedge extern' = extern \wedge externsel' = externsel \wedge build' = build \end{aligned} $	

AbortOuter

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = setting \wedge squeue(\#smode) \neq \langle \rangle$
 $head(squeue(\#smode)) \in \text{ran } badK \wedge sreproj(\#smode) = off$
 $fields' = ssnap(\#smode)$
 $smode' = front\ smode \wedge squeue' = front\ squeue$
 $\wedge ssnap' = front\ ssnap \wedge sprops' = front\ sprops$
 $\wedge sreproj' = front\ sreproj$
 $guard' = guard \wedge msel' = msel \wedge dataset' = dataset$
 $\wedge extern' = extern \wedge externsel' = externsel \wedge build' = build$

8 Delivering Notifications (reentrancy and order)

A committing frame delivers its props one at a time, in any order. Delivering a `ksel` notification is the model's `on_selection_gesture`: it folds the element's *committed* selection into the model's full selection:

DeliverSel

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = flushing \wedge ksel \in sprops(\#smode)$
 $msel' = fields\ ksel$
 $sprops' = sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{ksel\})\}$
 $smode' = smode \wedge squeue' = squeue$
 $\wedge ssnap' = ssnap \wedge sreproj' = sreproj$
 $fields' = fields \wedge dataset' = dataset \wedge extern' = extern$
 $\wedge externsel' = externsel \wedge guard' = guard \wedge build' = build$

Delivering a `krows` notification runs the model's rows observer. When the guard is *up*, the notification is the model's own re-projection write coming back: the observer returns early and folds nothing — the self-write guard:

DeliverRowsSkip

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = flushing \wedge krows \in sprops(\#smode)$
 $guard = on$
 $sprops' = sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{krows\})\}$
 $smode' = smode \wedge squeue' = squeue$
 $\wedge ssnap' = ssnap \wedge sreproj' = sreproj$
 $msel' = msel \wedge fields' = fields \wedge dataset' = dataset$
 $\wedge extern' = extern \wedge externsel' = externsel \wedge guard' = guard$
 $\wedge build' = build$

When the guard is *down*, the notification is an external dataset refresh and the model runs `_absorb_dataset`: it absorbs the dataset (`dataset` becomes the delivered rows value), *folds the element's committed selection* into its full selection ($msel' = fields\ ksel$ — the round-4 fix that makes the fold read post-patch state), raises the guard, and pushes a nested re-projection frame. That frame writes the filtered subset (b) into `krows` and the model's now-current selection ($fields\ ksel$) back into `ksel`. The nested frame carries its own empty buffer, and the outer frame's delivery set has already had `krows` removed, so the two buffers never share storage:

Reproject

ΔSys

$$\begin{aligned} \#smode &\neq 0 \\ smode(\#smode) &= flushing \wedge krows \in sprops(\#smode) \\ guard &= off \wedge \#smode < maxdepth \\ dataset' &= fields\ krows \wedge msel' = fields\ ksel \\ smode' &= smode \frown \langle setting \rangle \\ squeue' &= squeue \frown \langle \langle setK(krows, b), setK(ksel, fields\ ksel) \rangle \rangle \\ ssnap' &= ssnap \frown \langle fields \rangle \\ sprops' &= (sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{krows\})\}) \frown \langle \emptyset \rangle \\ sreproj' &= sreproj \frown \langle on \rangle \\ guard' &= on \wedge fields' = fields \\ \wedge extern' &= extern \wedge externsel' = externsel \wedge build' = build \end{aligned}$$

The same trigger may instead push a re-projection that *aborts* on its second key — the model asking “can the nested patch fail mid-flight?” Its rollback restores the fields to the post-outer-commit values, and the outer frame resumes flushing, so the outer commit is never left half-delivered:

ReprojectAbort

ΔSys

$$\begin{aligned} \#smode &\neq 0 \\ smode(\#smode) &= flushing \wedge krows \in sprops(\#smode) \\ guard &= off \wedge \#smode < maxdepth \\ dataset' &= fields\ krows \wedge msel' = fields\ ksel \\ smode' &= smode \frown \langle setting \rangle \\ squeue' &= squeue \frown \langle \langle setK(krows, b), badK\ ksel \rangle \rangle \\ ssnap' &= ssnap \frown \langle fields \rangle \\ sprops' &= (sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{krows\})\}) \frown \langle \emptyset \rangle \\ sreproj' &= sreproj \frown \langle on \rangle \\ guard' &= on \wedge fields' = fields \\ \wedge extern' &= extern \wedge externsel' = externsel \wedge build' = build \end{aligned}$$

When a frame has delivered every prop it is popped off every stack sequence. A re-projection frame lowers the guard; an outer frame leaves it untouched and, being the last frame, returns the element to idle:

PopReproj

ΔSys

$$\begin{aligned} \#smode &\neq 0 \\ smode(\#smode) &= flushing \wedge sprops(\#smode) = \emptyset \\ sreproj(\#smode) &= on \\ smode' &= front\ smode \wedge squeue' = front\ squeue \\ \wedge ssnap' &= front\ ssnap \wedge sprops' = front\ sprops \\ \wedge sreproj' &= front\ sreproj \\ guard' &= off \wedge fields' = fields \wedge msel' = msel \\ \wedge dataset' &= dataset \wedge extern' = extern \\ \wedge externsel' &= externsel \wedge build' = build \end{aligned}$$

PopOuter

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = flushing \wedge sprops(\#smode) = \emptyset$
 $sreproj(\#smode) = off$
 $smode' = front smode \wedge squeue' = front squeue$
 $\wedge ssnap' = front ssnap \wedge sprops' = front sprops$
 $\wedge sreproj' = front sreproj$
 $guard' = guard \wedge fields' = fields \wedge msel' = msel$
 $\wedge dataset' = dataset \wedge extern' = extern$
 $\wedge externsel' = externsel \wedge build' = build$

Every reachable non-idle state enables at least one operation: a **setting** top with a non-empty queue enables **ApplyStepTop** or an abort; with an empty queue, a commit; a **flushing** top with props to deliver enables **DeliverSel**, **DeliverRowsSkip**, or the re-projections; with nothing left, a pop. With **BeginOuter** enabled in every idle state, the correct design never deadlocks (Section 10).

9 The Invariant

Three safety clauses must hold whenever the element is idle ($\#smode = 0$). They are stated as a predicate over **Sys**; they are *not* placed in the **Sys** schema, because a predicate there would be enforced as a guard on every successor, silently disabling any operation that would break it and masking the very defect this model exists to catch.

Invariant AT — observable atomicity and order-independence. When no patch is in progress:

$$\#smode = 0 \Rightarrow (dataset = extern \wedge fields ksel = externsel \wedge msel = externsel).$$

The three conjuncts capture the whole contract:

- *Dataset* ($dataset = extern$). The model's dataset equals the dataset the agent last pushed — the last *external* **krows** write — not the filtered subset the model's own re-projection wrote back. Removing the round-3 guard breaks this: the model folds its own subset in.
- *Selection atomicity* ($fields ksel = externsel$). The element's committed selection equals the agent's patched selection. A round-1 (mutable snapshot) or round-2 (eager notify) defect breaks this — a rolled-back field left ahead, or an un-fired observer left ahead.
- *Order-independence* ($msel = externsel$). The model's full selection equals the agent's patched selection whichever notification the commit flushed first. Because **sprops** is a set delivered in any order, ProB reaches this idle state by every intra-batch order; the clause holding across all of them is order-independence. The round-4 defect — re-projecting off the stale full selection when **rows** flushes first — breaks it on exactly the rows-first order.

10 Verification

Type-checking is a **fuzz** pass:

```
fuzz -t docs/patch_atomicity.tex
```

Invariant AT is checked by asking ProB whether its negation is *reachable*; a goal that is not found means the property holds on every reachable state. The operation cap is raised because a single idle state offers many step-by-step-built outer patches:

```
# Invariant AT (must report: goal NOT found)
probccli docs/patch_atomicity.tex -model_check -nodead -p MAX_OPERATIONS 500 \
  -goal "size(smode)=0 & (dataset/=extern or fields(ksel)/=externsel
        or msel/=externsel)"
```

Deadlock-freedom is the state-space search with no goal:

```
# Deadlock-freedom (must report: no deadlock, no error)
probccli docs/patch_atomicity.tex -model_check -p MAX_OPERATIONS 500
```

The carriers are concrete free types, so both searches are already exhaustive — there is no `DEFAULT_SETSIZE` to raise. Robustness against the bounds is checked by re-running with `maxdepth` lowered to 2 (the correct design’s true maximum depth): the invariant still holds and no deadlock appears, confirming the depth-three headroom is only there to give the buggy variant room to recurse.

11 Fidelity: reproducing the four defects

A model that cannot reproduce the known defects proves nothing. Each variant is a single-schema edit; the model checker is re-run, and the named goal must now be *found* (or, for round 3, a deadlock must appear). Write k for `stepKey (head (squeue (#smode)))` and g for `stepVal (head (squeue (#smode)))`.

Round 2 — observer notified inside the patch. Model eager notification: in `ApplyStepTop`, instead of adding k to the frame’s `sprops` buffer, fold it immediately when it is the selection — if $k = ksel$ set $msel' = g$, leaving `sprops` unchanged so nothing is buffered. Now a patch that sets `ksel` then hits a `badK` folds the intermediate selection into `msel`, `AbortOuter` restores the field but cannot un-fold the model, and at idle $msel \neq externsel$: the AT goal is *found*.

Round 1 — snapshot aliases a mutable model. Model the shallow snapshot capturing a shared reference: in `ApplyStepTop` mutate the top frame’s snapshot alongside the field — $ssnap' = ssnap \oplus \{\#smode \mapsto (ssnap(\#smode) \oplus \{k \mapsto g\})\}$. Now a patch that sets `ksel` then hits a `badK` reaches `AbortOuter`, which restores $fields' = ssnap(\#smode)$ — but the snapshot was mutated with the field, so the field is not restored. At idle $fields ksel \neq externsel$: the AT goal is *found* from the opposite side.

Round 3 — the self-write guard removed. Delete `DeliverRowsSkip` and drop the `guard = off` conjunct from `Reproject` and `ReprojectAbort`, so every `krows` delivery re-absorbs and re-projects. The nested re-projection’s own `krows` write, delivered during its commit, is folded straight back in as the dataset — *dataset* becomes the subset `b` while the agent pushed `a` — and it pushes another re-projection until the depth bound. Two signals witness it:

```
# the observer folds its own subset in as the dataset
probccli docs/patch_atomicity.tex -model_check -nodead -p MAX_OPERATIONS 500 \
  -goal "guard=on & dataset/=extern" (* found *)
# the self-recursion, bounded only by the state space
probccli docs/patch_atomicity.tex -model_check -p MAX_OPERATIONS 500 (* deadlock *)
```

Round 4 — the rows-absorb reads the stale full selection. Model the pre-fix `_absorb_dataset` that re-projected off the stale full selection instead of folding the committed one: in `Reproject` and `ReprojectAbort`, drop the $msel' = fields ksel$ fold (set $msel' = msel$) and change the nested selection write from `setK (ksel, fields ksel)` to `setK (ksel, msel)` — the re-projection writes the *stale* model selection back into the element. Now a dual `{rows, selection}` patch whose commit delivers `rows` first re-projects the stale `msel` over the freshly-patched selection, the later `selection` delivery reads the clobbered value, and at idle $fields ksel \neq externsel$ (and $msel \neq externsel$): the AT goal is *found*. The selection-first order still satisfies AT, so the reachability of the violation is precisely the order-dependence the round-4 fix removed:

```
probccli docs/patch_atomicity.tex -model_check -nodead -p MAX_OPERATIONS 500 \
  -goal "size(smode)=0 & fields(ksel)/=externsel" (* found *)
```

Under the correct design none of the four traces exists. `ApplyStepTop` defers every notification into the frame’s own buffer and leaves the snapshot untouched; the aborts restore a value snapshot and fire nothing; `DeliverRowsSkip` stops the model re-absorbing its own re-projection; and `Reproject` folds the committed selection before writing it back. The AT search returns *goal not found*, the round-3 and round-4 goals are unreachable, and the machine is deadlock-free.

12 Nested-case partitions and test coverage

The reentrancy adds five partitions to the single-patch atomicity partitions the earlier rounds already covered. Each is a reachable class of trace in the model; the table names the operation sequence, the AT outcome, and the test in `tests/test_table_composition.py` that exercises it.

Partition (model trace)	AT outcome	Covering test
External rows refresh under a filter: outer rows patch, commit, <code>Reproject</code> , nested subset write, <code>DeliverRowsSkip</code> (guard up), pop.	$dataset = extern$ (full dataset, not subset).	<code>test_agent_rows_patch_refreshes_the_dataset_under</code>
Self-write guard holds: the nested re-projection's own <code>krows</code> delivery is skipped, not re-absorbed.	Dataset stays the full set; round-3 goal unreachable.	<code>test_the_models_own_reprojection_keeps_the_data</code>
Dataset change reconciles the full selection (ids gone from the data leave it), distinct from a filter hiding them.	Selection folds to the reconciled set.	<code>test_agent_rows_patch_reconciles_the_full_select</code>
Dual rows+selection patch, <i>either</i> delivery order: rows-first folds the committed selection before re-projecting; selection-first folds it directly.	$fields\ ksel = externsel$ and $msel = externsel$ both orders (order-independent).	<code>test_dual_rows_and_selection_patch_keeps_the_se</code>
Nested re-projection <i>aborts</i> mid-flight (<code>ReprojectAbort</code>); the outer commit still delivers its remaining props.	Outer never half-delivered; all clauses hold.	<i>Gap</i> — covered in the model, not in a test.

The single-patch atomicity partitions remain covered by `test_a_failed_multi_key_patch_leaves_the_model_unc` and `test_a_successful_patch_still_folds_through_to_the_model` in `TestWriteThroughAtomicity`.

The one gap. The `ReprojectAbort` partition — a nested re-projection that raises after writing its first key — is proven safe by the model (the outer commit finishes delivering, and every AT clause holds at idle) but has no dedicated test. In the shipped code a re-projection writes only validated subset rows and a projected selection, so it does not raise in practice; the partition guards against a future setter that could. A regression test that forces the model's inner `apply_patch` to reject its second key and asserts the outer selection notification still lands would close it. The model stands as the regression artefact for the interleaving until then.